Informe post-incidente — TP Unidad 6: respaldo y recuperación de Food Store

Bases de Datos II · Grupo 11 · Emmanuel Miranda · Simulacro: 28/10/2026 · Entorno: PostgreSQL 16.15 local de laboratorio, `foodstore` cargada con `parcial_BD/01_schema` + `04_objects` + `03_data_masivo` (50k productos, 200k pedidos, 400k detalles).

Dato	Valor
Respaldo utilizado	`foodstore_20261028_222938.dump`, tomado el **28/10/2026 22:29:38 (-03)** · custom (`-Fc`) · 4,9 MB · 78 objetos en el TOC · sha256 `4fef905a…`
Tiempo de restauración y validación	`createdb` 22:29:55 → validaciones OK 22:29:58: **3,0 s** (de esos, `pg_restore` 2,7 s)

RPO y RTO. El RPO mide **cuántos datos se pierden**: lo escrito desde el último respaldo hasta la falla. El RTO mide **cuánto se tarda en volver a operar**.

	Previsto (SPEC_U6)	Observado
RPO	24 h (respaldo diario)	Antigüedad del respaldo al empezar: **17 s**. Pérdida real: **5 pedidos** cargados a las 22:29:50, después del respaldo, que no están en la copia restaurada (200.005 pedidos en el original contra 200.000 en la copia). En producción la ventana puede llegar a 24 h de ventas.
RTO	15 min	**3,0 s** (más de 200 veces por debajo del objetivo, por el volumen chico del laboratorio)

Validaciones sobre `foodstore_restore_test`. Conteos idénticos al registro previo al respaldo (`.counts`): categoria 12 · usuario 20.000 · producto 50.000 · pedido 200.000 · detalle_pedido 400.000. `productos_invalidos = 0` y `detalles_invalidos = 0`. Controles extra: 0 FKs huérfanas; 18 restricciones, 13 índices, 7 vistas/MV, 6 funciones y 8 triggers, igual que en el original. `foodstore` no se tocó en ningún momento.

Problema real encontrado. La validación de referencia del PDF cuenta la tabla `cliente`, que **no existe** en nuestro schema (los clientes son filas de `usuario`). Si se copiaba tal cual, el simulacro fallaba en la validación y el cronómetro no cerraba. Se corrigió en `conteo_filas.sql` y quedó registrado en `AGENTS.md`. Además, el chequeo de salud encontró: (1) `mv_top_productos_categoria` **vacía** con ventas cargadas, porque `04_objects` corre antes del seed (se resolvió con `REFRESH … CONCURRENTLY` en `maintenance.sql`); (2) `pedido` con **74 % de espacio libre** (bloat): el trigger de total hace un UPDATE por cada detalle durante la carga. No se usó `VACUUM FULL` periódico: la restauración lógica lo eliminó (`pedido` pasó de 119 MB a 27 MB). (3) El seed masivo asigna el mismo producto y la misma cantidad a todos los detalles: los `LATERAL (SELECT random())` sin correlación se evalúan una sola vez. Queda asentado para corregirlo en el seed.

Mejora concreta. Restaurar en paralelo con `pg_restore --jobs=N` (N = núcleos; el script ya lo acepta con `JOBS=4`). Medido sobre el mismo dump, 3 corridas: secuencial 4,1 / 3,3 / 3,3 s; paralelo 2,7 / 3,1 / 2,7 s. **La mediana baja de 3,3 a 2,7 s (−18 %)** con sólo 2 núcleos, y la ganancia crece con bases más grandes. Agregar `vacuumdb --analyze-in-stages` al final del restore: `pg_restore` no trae estadísticas y la copia arranca con planes malos. Para bajar el RPO por debajo de 24 h hace falta archivado de WAL (PITR), no más dumps.

Trabajo asistido con IA. Las primeras versiones de `SPEC_U6.md`, los dos scripts `.sh`, `health_check.sql` y `maintenance.sql` se generaron con un asistente de IA a partir de la consigna, `SPEC_U6.md` y `AGENTS.md`. Ningún script se aceptó sin correrlo contra la base, y las pruebas detectaron estos errores de la IA: el `ANALYZE` de la MV estaba **antes** del `REFRESH` y dejaba estadísticas vacías; la prueba de falla del backup cortaba en `psql` y **no ejercía** `pg_dump` (se agregó un rol sin permiso sobre las secuencias, y el script terminó con código 1 sin dejar archivos parciales); dos corridas en el mismo segundo pisaban el mismo log; y se usaba `bc`, que Git Bash no trae. Ningún script contiene contraseñas: se usa `.pgpass` con `--no-password`. Evidencia completa en `evidencias/01` a `04`.